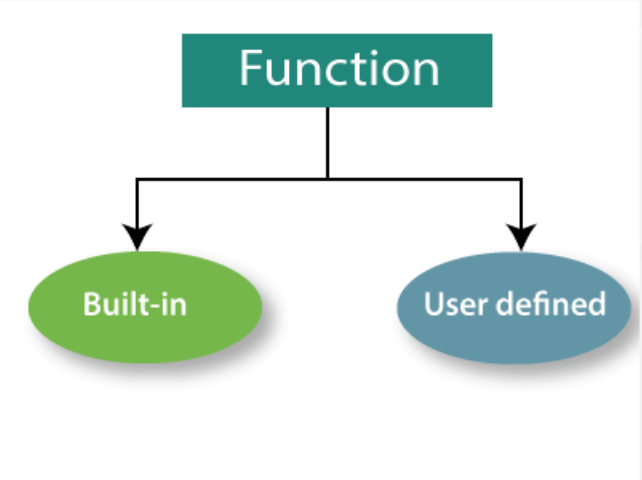
R

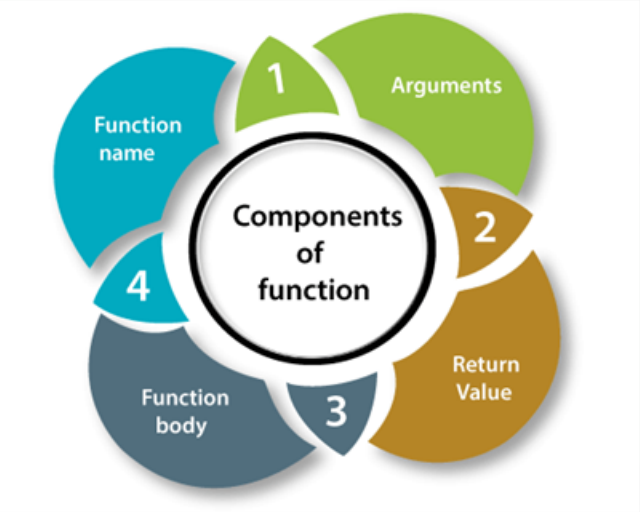
Functions

Objective

- Functions
- Function Types
- Built in functions
- Numeric functions
- Character Function
- Summary

Functions





An R function is created by using the keyword function. There is the following syntax of R function:

func_name <- function(arg_1, arg_2) {
Function body
}

```
add<-function(a,b,c){
  a+b+c
}
add(10,20,30)
```

```
addition<-function(a,b){
  return(a+b)
}
addition(5,7)
```


www.object.co.in

Function Types

- **Basic Functions:** Simple input-output operations.
- **Parameterized Functions:** With or without default values.
- **Default Parameter Functions:** Functions can have default values for parameters. If a parameter is not passed, the default value is used.
- **Named Parameter Functions:** arguments can be explicitly referenced by their names.
- **Variable Arguments**: Handle a flexible number of inputs.

```
sum_numbers <- function(...){
  numbers <- c(...)  # Combine arguments into a vector
  return(sum(numbers))
}

sum_numbers(10, 20, 30, 40)
```


www.object.co.in

Imp Built in functions

- **str**

str displays structures of R objects. str are mostly used for displaying the contents of a list.

```
lst<-list(1,2,3,4,5,6,3,4,5,6)
str(lst)
```

- **seq**

seq() function in R Language is used to create a sequence of elements in a Vector. It takes the length and difference between values as optional argument.

Syntax:

seq(from, to, by, length.out)

Parameters:

from: Starting element of the sequence
to: Ending element of the sequence
by: Difference between the elements
length.out: Maximum length of the vector

```
# Creating vector using seq()
vec1 <- seq(1, 10, by = 2)
vec2 <- seq(1, 10, length.out = 7)

print(vec1) #1 3 5 7 9
print(vec2) #1.0 2.5 4.0 5.5 7.0
8.5 10.0
```

- table
  table() function in R Language is used to create a categorical representation of data with variable name and the frequency in the form of a table.
  Syntax: table(x)

```
# Creating a vector
vec = c(2, 4, 3, 1, 2, 3, 2, 1, 4, 2)
# Calling table() Function
table(vec)
o/p
vec
1 2 3 4
2 4 2 2
```

- log
  log() function in R Language returns the natural logarithm (base-e logarithm) of the argument passed in the parameter.
  Syntax: log(x)
  Parameter:
  x: Specified value.
  Returns: It returns the natural logarithm of the specified value, infinity for 0 and NaN for the negative value.

```
print(log(1))
print(log(30))
```


www.object.co.in

- • log computes logarithms, by default natural logarithms,
  • log10 computes common (i.e., base 10) logarithms,
  • log2 computes binary (i.e., base 2) logarithms.
  • The general form log(x, base) computes logarithms with base base.

- **exp**
  exp computes the exponential function.

```
> log(2)
[1] 0.6931472
> log(2,10)
[1] 0.30103
> log1p(2)
[1] 1.098612
> log10(2)
[1] 0.30103
> log2(2)
[1] 1
```

```
> exp(0.6931472)
[1] 2
> 10^0.30103
[1] 2
> expm1(1.098612)
[1] 1.999999
> 10^0.30103
[1] 2
> 2^1
[1] 2
```


www.object.co.in

- ifelse

  This is a shorthand function to the traditional if…else statement.

  Syntax : ifelse(condition, true-value, false-value)

  If the condition is TRUE then the function returns true-value otherwise it returns false-value.

```
v<-c(1,2,3,4)
ifelse(v%%2==0,"even","odd")
o/p
"odd"   "even"  "odd"   "even"
```

- Attach

  attach() function in R Language is **used to access the variables present in the data framework without calling the data frame**.

  Syntax:

  attach(data)

  detach(data)

```
data <- data.frame(x1 = c(9, 8, 3, 4, 8),
                   x2 = c(5, 4, 7, 1, 1),
                   x3 = c(1, 2, 3, 4, 5))
data
data$x1
attach(data)
x1
detach(data)
o/p:
  > data$x1
[1] 9 8 3 4 8
> attach(data)
> x1
[1] 9 8 3 4 8
```


www.object.co.in

Mean and Variance Functions

- **mean()**

To compute the average of values, R provides a pre-defined function mean(). This function takes a Numerical Vector as an argument and results in the average/mean of that Vector.

- **sd()**

sd() function is used to compute the standard deviation of given values in R. It is the square root of its variance.

- var()

var() function in R Language computes the sample variance of a vector. It is the measure of how much value is away from the mean value.

- Each of the functions above have the syntax usage in the following way:
  function-name(variable-name,na.rm) – na.rm by default is FALSE. It should be set to TRUE if we want to ignore the NA values while computing.

```
list = c(2, 4, 4, 4, 5, 5, 7, 9)

print(mean(list))
print(sd(list))
print(var(list))
o/p:
  print(mean(list))
[1] 5
> print(sd(list))
[1] 2.13809
> print(var(list))
[1] 4.571429
```

- summary() function in R Language is a generic function used to produce result summaries of the results of various model fitting functions.

```
# Initializing a character and
# integer vector
x <- c("snehal", "avani", "snehal", "arnav")
y <- c(1, 2, 3, 4)

# Calling summary() function
summary(x)
summary(y)

o/p:
summary(x)
Length     Class      Mode
4 character character
> summary(y)
Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
1.00    1.75    2.50    2.50    3.25    4.00
```


www.object.co.in

Numeric functions

```
 abs(-4)
[1] 4

> sqrt(64)
[1] 8

> floor(12.3)
[1] 12

> round(12.3)
[1] 12

> ceiling(12.3)
[1] 13
```


www.object.co.in

Function	Description	Example	Output
nchar(x)	Counts number of characters in a string	nchar("hello")	5
toupper(x)	Converts all letters to uppercase	toupper("hello")	"HELLO"
tolower(x)	Converts all letters to lowercase	tolower("WORLD")	"world"
substr(x, start, stop)	Extracts or replaces substring from position start to stop	substr("example", 2, 4)	"xam"
substring(x, first, last)	Similar to substr(), can be vectorized	substring("data", 2)	"ata"
strsplit(x, split)	Splits string into parts based on a delimiter	strsplit("a-b-c", "-")	[[1]] "a" "b" c"
grep(pattern, x)	Returns indices of matches (pattern matching)	grep("a", c("apple", "banana"))	1 2
grepl(pattern, x)	Logical: TRUE if pattern is found	grepl("x", "text")	TRUE
gsub(pattern, replacement, x)	Replaces all matches of pattern	gsub("cat", "dog", "catnap")	"dognap"

Character Function

```
s<-"Happy Journey"
> nchar(s) #Count the Number of Characters
[1] 13

> toupper(s)
[1] "HAPPY JOURNEY“

> tolower(s)
[1] "happy journey“

> substr(s,2,5) #substr(x, start, stop)Substrings of a Character Vector
[1] "appy“
```


www.object.co.in

```
> gsub("p","z",s)#Pattern Matching and Replacement sub and gsub perform replacement of the first and all matches respectively.
[1] "Hazzy Journey"
>
> strsplit(s," ") #Split the Elements of a Character Vector
[[1]]
[1] "Happy"   "Journey"
```


www.object.co.in

Summary

- Functions
- Function Types
- Built in functions
- Numeric functions
- Character Function
- Summary

www.object.co.in

OBJECT™ TECHNOLOGIES

Thank You